

# Tuần 3 — **DynamoDB** toàn tập

**Domain:** Domain 1 – Development with AWS Services (32%) · **Thời lượng:** ~11h (4 buổi) · **Vị trí:** Tuần 3/5 của Domain 1

**Điều hướng:**  [Tuần 2](#) ·  [Kế hoạch tổng](#) · [Tuần 4](#) 

## Mục tiêu tuần này

- **Tính được** WCU/RCU cho ít nhất 5 kịch bản đọc/ghi (strong vs eventual, item bất kỳ kích thước) không cần tra bảng.
- **Giải thích được** khác biệt GSI vs LSI và **chọn đúng** loại index theo tình huống đề.
- **Tự tay** tạo bảng có partition key + sort key, thêm 1 GSI + 1 LSI, bật **DynamoDB Streams** → trigger **Lambda**.
- **Viết được** conditional write + optimistic locking (version number) và **TransactWriteItems** bằng SDK/CLI.
- **Phân biệt** Query vs Scan, dùng đúng pagination (**LastEvaluatedKey**) và filter expression.
- **Nhận diện** dấu hiệu hot partition + throttling và biết cách xử lý (**DAX**, retry backoff, thiết kế key).

## Nội dung học chi tiết

### Buổi A — Lý thuyết (~3h)

#### 1. Data model & giới hạn item

- **DynamoDB** là NoSQL key-value + document. Primary key có 2 kiểu:
  - **Partition key (hash key)** đơn: giá trị được băm để chọn partition.
  - **Partition key + Sort key (range key)** (composite): cùng partition key gom vào 1 partition, sắp theo sort key → cho phép Query theo dải.
- **Item tối đa 400 KB** (gồm cả tên attribute + giá trị). Item lớn → cân nhắc lưu ở **S3** và giữ pointer trong item.

- **Vì sao quan trọng với đề:** đề hay hỏi "item 400KB có lưu được không", "chọn key nào để query hiệu quả", "vì sao Scan chậm".

## 2. Capacity & throughput — tính WCU/RCU (CỐT LỖI, hỏi nhiều)

- **1 WCU** = 1 write/giây cho item  $\leq 1$  KB. Item lớn hơn → làm tròn LÊN bội số 1 KB.
- **1 RCU** = **1 strongly consistent read/giây** cho item  $\leq 4$  KB; **HOẶC 2 eventually consistent reads/giây** cho item  $\leq 4$  KB. Item lớn hơn → làm tròn LÊN bội số 4 KB.
- **Transactional read = 2× RCU; transactional write = 2× WCU.** Mỗi transaction tối đa **100 action / 100 item / 4 MB**.
- **Ví dụ tính (thuộc lòng cách làm):**

| Kịch bản                                                     | Cách tính                                                       | Kết quả        |
|--------------------------------------------------------------|-----------------------------------------------------------------|----------------|
| Đọc item <b>8 KB, strongly consistent</b> , 10 lần/s         | mỗi read = $\text{ceil}(8/4)=2$ RCU<br>→ $\times 10$            | <b>20 RCU</b>  |
| Cũng item 8 KB nhưng <b>eventually consistent</b> , 10 lần/s | eventual chia đôi → $20/2$                                      | <b>10 RCU</b>  |
| Ghi item <b>3 KB</b> , 6 lần/s                               | mỗi write = $\text{ceil}(3/1)=3$ WCU<br>→ $\times 6$            | <b>18 WCU</b>  |
| Đọc item <b>1 KB</b> , strongly, 100 lần/s                   | $\text{ceil}(1/4)=1$ RCU → $\times 100$                         | <b>100 RCU</b> |
| <b>Transactional</b> write item 2 KB, 5 lần/s                | $\text{ceil}(2/1)=2$ WCU $\times 2$<br>(transaction) $\times 5$ | <b>20 WCU</b>  |

- **On-demand vs Provisioned:**

- **On-demand:** tự co giãn, trả theo request — hợp workload không dự đoán được / spiky / mới.
- **Provisioned:** đặt trước WCU/RCU (rẻ hơn khi tải ổn định) + bật **Auto Scaling** để tự điều chỉnh theo target utilization.
- **Lưu ý:** on-demand hiện là **mặc định** & được AWS **khuyến nghị** cho hầu hết workload (không cần capacity planning).

## 3. Query vs Scan

- **Query:** truy vấn theo **partition key** (bắt buộc) + tùy chọn điều kiện trên **sort key**. Nhanh, chỉ đọc phần dữ liệu liên quan → **luôn ưu tiên Query**.

- **Scan**: đọc **toàn bộ bảng** rồi lọc → tốn RCU, chậm. Chỉ dùng khi bắt buộc; có thể bật **Parallel Scan** để chia segment.
- **Filter expression**: áp dụng **SAU** khi đọc dữ liệu → **vẫn tính RCU trên dữ liệu đã đọc**, không tiết kiệm capacity, chỉ giảm dữ liệu trả về.
- **Pagination**: Query/Scan trả **tối đa 1 MB/lần**. Nếu còn dữ liệu → trả về **LastEvaluatedKey**; truyền lại vào **ExclusiveStartKey** để lấy trang tiếp.

#### 4. Index — GSI vs LSI

| Tiêu chí      | GSI (Global Secondary Index)          | LSI (Local Secondary Index)                   |
|---------------|---------------------------------------|-----------------------------------------------|
| Partition key | <b>Khác</b> bảng gốc                  | <b>Cùng</b> partition key với bảng            |
| Sort key      | Khác (tùy chọn)                       | <b>Khác</b> sort key                          |
| Throughput    | <b>RIÊNG</b> (WCU/RCU riêng)          | <b>DÙNG CHUNG</b> với bảng gốc                |
| Consistency   | <b>CHỈ eventually consistent</b>      | <b>Hỗ trợ strongly consistent</b>             |
| Thời điểm tạo | <b>Bất kỳ lúc nào</b> (tạo/xoá tự do) | <b>BẮT BUỘC</b> lúc tạo bảng (không thêm sau) |
| Giới hạn      | mặc định <b>20 GSI/bảng</b>           | <b>Tối đa 5 LSI/bảng</b>                      |

- **Chọn khi nào**: cần query theo **thuộc tính khác partition key** → **bắt buộc GSI** (LSI không đổi được partition key). Cần **strongly consistent read** trên index + cùng partition key → LSI.

## Buổi B — Hands-on (~3.5h)

 **Lab cầm tay chỉ việc (từng bước + lệnh + code):** [labs.md](https://labs.md).

### Bước 1 — Tạo bảng có composite key + LSI (LSI phải khai lúc tạo):

```
aws dynamodb create-table \
  --table-name Orders \
  --attribute-definitions \
    AttributeName=CustomerId,AttributeType=S \
    AttributeName=OrderDate,AttributeType=S \
    AttributeName=Amount,AttributeType=N \
  --key-schema \
    AttributeName=CustomerId,KeyType=HASH \
    AttributeName=OrderDate,KeyType=RANGE \
  --local-secondary-indexes \
```

```
"IndexName=ByAmount,KeySchema=[{AttributeName=CustomerId,KeyType=HASH},
{AttributeName=Amount,KeyType=RANGE}],Projection={ProjectionType=ALL}" \
--billing-mode PAY_PER_REQUEST
```

## Bước 2 — Thêm GSI SAU khi bảng đã tồn tại (chứng minh GSI tạo bất kỳ lúc nào):

```
aws dynamodb update-table --table-name Orders \
  --attribute-definitions AttributeName=Status,AttributeType=S \
  --global-secondary-index-updates \
  '[{"Create":{"IndexName":"ByStatus","KeySchema":
[{"AttributeName":"Status","KeyType":"HASH"}],"Projection":
{"ProjectionType":"ALL"}}}]'
```

## Bước 3 — Bật **DynamoDB Streams** và trigger **Lambda**:

- Bật Streams với view type mong muốn (**NEW\_AND\_OLD\_IMAGES** để thấy cả trước/sau).

```
aws dynamodb update-table --table-name Orders \
  --stream-specification
StreamEnabled=true,StreamViewType=NEW_AND_OLD_IMAGES
```

- Tạo **event source mapping** để **Lambda** đọc stream (nhớ role Lambda cần quyền đọc stream):

```
aws lambda create-event-source-mapping \
  --function-name ProcessOrderStream \
  --event-source-arn <stream-arn> \
  --starting-position LATEST
```

## Bước 4 — Conditional write + optimistic locking (version number):

```
# Chỉ ghi nếu item CHƯA tồn tại (chống ghi đè)
aws dynamodb put-item --table-name Orders \
  --item '{"CustomerId":{"S":"C1"},"OrderDate":{"S":"2026-01-01"},"Version":
{"N":"1"}}' \
  --condition-expression "attribute_not_exists(CustomerId)"

# Optimistic locking: chỉ update nếu Version còn = giá trị mình đã đọc
aws dynamodb update-item --table-name Orders \
  --key '{"CustomerId":{"S":"C1"},"OrderDate":{"S":"2026-01-01"}}' \
  --update-expression "SET Amount = :a, Version = :newv" \
  --condition-expression "Version = :curv" \
```

```
--expression-attribute-values '{"a":{"N":"500"},"newv":
{"N":"2"},"curv":{"N":"1"}}'
```

- Nếu điều kiện sai → trả về **ConditionalCheckFailedException** (không tốn ghi đè). Đây là nền tảng của **atomic counters** (**SET x = x + :inc**) và tránh race condition.

## Bước 5 — **TransactWriteItems** (all-or-nothing, ACID trên nhiều item/bảng):

```
aws dynamodb transact-write-items --transact-items '[
  {"Put":{"TableName":"Orders","Item":{"CustomerId":{"S":"C2"},"OrderDate":
{"S":"2026-02-01"}}}},
  {"Update":{"TableName":"Inventory","Key":{"Sku":{"S":"SKU1"}}},
    "UpdateExpression":"SET Qty = Qty - :n","ConditionExpression":"Qty >=
:n",
    "ExpressionAttributeValues":{"n":{"N":"1"}}}}
]
```

**Bước 6 — TỰ TÍNH WCU/RCU cho 5 kịch bản** (dùng lại bảng ví dụ ở Buổi A, tự bấm tay từng bước ceil + strong/eventual + transaction x2 rồi đối chiếu).

## **C** Buổi C — Bổ sung (~2.5h)

- **DynamoDB Streams** sâu: giữ record **24 giờ**; 4 view type: **KEYS\_ONLY** (chỉ key) / **NEW\_IMAGE** (item sau thay đổi) / **OLD\_IMAGE** (item trước) / **NEW\_AND\_OLD\_IMAGES** (cả hai). Dùng cho CDC, cập nhật search index, gửi notification.
- **DAX (DynamoDB Accelerator)**: cache **độ trễ micro-giây**, chạy **in-VPC**, **write-through**, chủ yếu tăng tốc **read** (đặc biệt read-heavy, repeated reads): **chỉ cache eventually consistent read**; strong read = pass-through không cache; không hợp app write-heavy/cần strong read. Không thay thế cache cho aggregation phức tạp; không giúp write.
- **Batch & TTL & PartiQL**:
  - **BatchGetItem / BatchWriteItem**: gom nhiều thao tác trong 1 call (không transaction — không all-or-nothing; item lỗi trả về **UnprocessedItems** để retry).
  - **TTL**: dựa trên **attribute dạng epoch (giây)**; item hết hạn bị xóa **trong vài ngày** sau khi hết hạn (không tức thì; không tốn WCU) — *tài liệu/để cũ hay ghi ~48h*. Dùng cho session, log tạm.

- **PartiQL**: cú pháp SQL-like (**SELECT/INSERT/UPDATE/DELETE**) để thao tác

**DynamoDB** — tiện nhưng vẫn theo cơ chế key bên dưới.

- **Chống hot partition & throttling**: đọc FAQ về key design — chọn partition key **có tính phân tán cao** (tránh dồn traffic vào 1 giá trị). Thêm **write sharding** (suffix ngẫu nhiên/hash) cho key nóng. Throttling →

**ProvisionedThroughputExceededException** → SDK tự retry với **exponential backoff**.

## **D** Buổi D — Practice + Review (~2h)

 **Bộ câu hỏi luyện tập của tuần:** [questions.md](#) — đáp án & giải thích: [answers.md](#). (bằng tiếng Anh — văn phong đề thật để làm quen đề.)

- Làm **20–30 câu practice** chủ đề **DynamoDB** (tính capacity, GSI/LSI, Streams, conditional write). Bấm tay lại các bài tính WCU/RCU cho tới khi phản xạ.
- **Ghi sổ câu sai**: đề + đáp án đúng + **lý do mình sai**. Đặc biệt các câu bẫy về consistency và index.
- **Spaced repetition**: ôn lại theo mốc **1 ngày → 3 ngày → 7 ngày** (đặc biệt bảng công thức RCU/WCU và bảng GSI vs LSI).
- Trả lời trôi chảy **Cổng tự kiểm tra** bên dưới trước khi sang Tuần 4.

## **PHẢI NHỚ** tuần này

| Fact                 | Con số / Quy tắc                                                                                          |
|----------------------|-----------------------------------------------------------------------------------------------------------|
| Item tối đa          | <b>400 KB</b>                                                                                             |
| 1 WCU                | 1 write/s cho item $\leq$ <b>1 KB</b> (làm tròn LÊN bội số 1 KB)                                          |
| 1 RCU<br>(strong)    | 1 strongly consistent read/s cho item $\leq$ <b>4 KB</b>                                                  |
| 1 RCU<br>(eventual)  | <b>2</b> eventually consistent reads/s cho item $\leq$ 4 KB                                               |
| Transaction          | read = <b>2× RCU</b> ; write = <b>2× WCU</b> ; mỗi transaction tối đa <b>100 action / 100 item / 4 MB</b> |
| Query/Scan<br>trả về | tối đa <b>1 MB/lần</b> → phải paginate bằng <b>LastEvaluatedKey</b>                                       |

| Fact       | Con số / Quy tắc                                                                                                                        |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| GSI        | key khác bảng, <b>throughput riêng, chỉ eventual</b> , tạo/xoá <b>bất kỳ lúc nào</b>                                                    |
| LSI        | <b>cùng partition key</b> , khác sort key, <b>chung throughput, hỗ trợ strong, tạo lúc tạo bảng</b> , tối đa <b>5/bảng</b>              |
| Streams    | giữ <b>24 giờ</b> ; view:<br><b>KEYS_ONLY/NEW_IMAGE/OLD_IMAGE/NEW_AND_OLD_IMAGES</b>                                                    |
| <b>DAX</b> | cache <b>micro-giây</b> , in-VPC, <b>write-through</b> , tăng tốc <b>read (chỉ eventual read)</b> ; strong = pass-through, không cache) |
| Throttling | <b>ProvisionedThroughputExceededException</b> → SDK retry<br><b>exponential backoff</b>                                                 |
| TTL        | attribute <b>epoch</b> ; xoá <b>trong vài ngày</b> sau khi hết hạn (không tức thì; không tốn WCU) — <i>để cũ hay ghi ~48h</i>           |

## ⚠ Bẫy đề hay gặp

- Thấy "**query theo thuộc tính khác partition key**" → dễ chọn LSI, nhưng đúng là **GSI** (LSI không đổi được partition key).
- Thấy "**cần thêm index sau khi bảng đã chạy**" → chọn LSI là **SAI**; LSI phải tạo lúc tạo bảng → đúng là **GSI**.
- Thấy "**strongly consistent read trên index**" → GSI **không** làm được (chỉ eventual) → phải là **LSI**.
- Thấy "**filter expression giúp tiết kiệm RCU**" → **SAI**: filter chạy sau khi đọc, **vẫn tốn RCU** trên dữ liệu đã quét.
- Thấy **eventually consistent** trong bài tính RCU → nhớ **chia đôi** số RCU so với strongly consistent.
- Thấy "**tránh ghi đè khi tạo mới / race condition**" → **conditional write** (**attribute\_not\_exists**) hoặc **optimistic locking** bằng version, **KHÔNG** phải transaction cho case đơn giản.
- Thấy "**nhiều thao tác phải cùng thành công hoặc cùng thất bại**" → **TransactWriteItems** (nhớ **2× WCU**), **KHÔNG** phải BatchWriteItem (batch không all-or-nothing).
- Thấy "**tăng tốc đọc micro-giây, read-heavy**" → **DAX**, không phải **ElastiCache** (để DynamoDB ưu tiên DAX).

- Thấy "**dữ liệu tự hết hạn**" → **TTL**, nhưng nhớ xoá **trong vài ngày** sau khi hết hạn (không tức thì; không tốn WCU) — *để cũ hay ghi ~48h* — nếu đề cần chính xác thời điểm thì phải lọc bằng expression.



## Phản xạ nhanh (keyword → đáp án)

| Thấy từ khoá                                      | Bật ngay                                                                        |
|---------------------------------------------------|---------------------------------------------------------------------------------|
| "query theo attribute khác key gốc"               | <b>GSI</b>                                                                      |
| "thêm index sau khi bảng đã tồn tại"              | <b>GSI</b> (LSI không được)                                                     |
| "strongly consistent read trên index"             | <b>LSI</b>                                                                      |
| "toàn bảng chậm / đọc hết bảng"                   | <b>Scan</b> (nên đổi sang <b>Query</b> )                                        |
| "kết quả bị cắt / còn dữ liệu"                    | pagination<br>bằng** <b>LastEvaluatedKey</b> **                                 |
| "tránh ghi đè / cập nhật khi chưa đổi"            | <b>conditional write / optimistic locking (version)</b>                         |
| "đếm tăng giảm an toàn"                           | <b>atomic counter</b> ( <b>SET x = x + :n</b> )                                 |
| "nhiều thao tác all-or-nothing"                   | <b>TransactWriteItems</b> (2× WCU)                                              |
| "cache đọc micro-giây, in-VPC"                    | <b>DAX</b>                                                                      |
| "phản ứng khi item thay đổi / trigger Lambda"     | <b>DynamoDB Streams</b> + event source mapping                                  |
| "tự xoá dữ liệu hết hạn"                          | <b>TTL</b> ( <b>trong vài ngày</b> , không tức thì; <i>để cũ hay ghi ~48h</i> ) |
| " <b>ProvisionedThroughputExceededException</b> " | <b>retry exponential backoff /</b> tăng capacity / chống hot partition          |
| "SQL-like trên DynamoDB"                          | <b>PartiQL</b>                                                                  |



## Lab checklist

- ☐ Tạo bảng **Orders** (partition + sort key) kèm **1 LSI** ngay lúc tạo bảng.
- ☐ Thêm **1 GSI** vào bảng đã tồn tại bằng **update-table**.
- ☐ Bật **DynamoDB Streams** (**NEW\_AND\_OLD\_IMAGES**) và gắn **event source mapping** → **Lambda**.
- ☐ Thực hiện **conditional write** (**attribute\_not\_exists**) và **optimistic locking** bằng version.
- ☐ Chạy **TransactWriteItems** trên  $\geq 2$  item/bảng với condition.
- ☐ Chạy **1 Query** + **1 Scan**, quan sát pagination **LastEvaluatedKey**.
- ☐ **Tự tính WCU/RCU** cho đủ 5 kịch bản và đối chiếu bảng.

## **Cổng tự kiểm tra (phải trả lời trôi chảy mới sang tuần sau)**

1. **1 RCU đọc được bao nhiêu KB, strong vs eventual? Đáp án gọn:** 1 RCU = 1 strongly consistent read cho item  $\leq 4$  KB/s; hoặc 2 eventually consistent reads  $\leq 4$  KB/s. Item lớn hơn làm tròn LÊN bội số 4 KB.
2. **Tính RCU: đọc item 8 KB strongly consistent 10 lần/s? Nếu eventual thì sao? Đáp án gọn:**  $\text{ceil}(8/4)=2$  RCU/read → **20 RCU**; eventual chia đôi → **10 RCU**.
3. **Khi nào BẮT BUỘC dùng GSI thay vì LSI? Đáp án gọn:** khi cần query theo **partition key khác** bảng gốc, hoặc cần **thêm index sau khi bảng đã tạo** (LSI phải tạo lúc tạo bảng).
4. **Query khác Scan ở chỗ nào? Đáp án gọn:** Query đọc theo partition key (+sort key) chỉ phần liên quan → nhanh, ít RCU; Scan đọc **toàn bảng** rồi lọc → chậm, tốn RCU. Luôn ưu tiên Query.
5. **Cách tránh hot partition? Đáp án gọn:** chọn partition key phân tán cao, **write sharding** (thêm suffix hash/ngẫu nhiên) cho key nóng, cân nhắc on-demand; kết hợp **DAX** giảm áp lực read.
6. **Ghi item 3 KB, 6 lần/s cần bao nhiêu WCU? Đáp án gọn:**  $\text{ceil}(3/1)=3$  WCU/ghi →  $\times 6 =$  **18 WCU**.

## **Tài nguyên tuần này**

 **Đã crawl sẵn tài liệu AWS vào [resources/](#) — đọc offline được.**

- AWS Docs: **Amazon DynamoDB Developer Guide** — mục *Read/Write Capacity Mode, Working with Queries & Scans, Secondary Indexes (GSI/LSI)*.

- AWS Docs: **DynamoDB Developer Guide** — *Change Data Capture with Streams, DAX, Time to Live (TTL), Working with Transactions, PartiQL for DynamoDB*.
- AWS FAQ: **Amazon DynamoDB FAQs** (capacity, consistency, throttling, best practices key design).
- Khoá **Stephane Maarek** — section **DynamoDB** (WCU/RCU, indexes, streams, DAX, transactions).
- Khoá **Adrian Cantrill** — phần **DynamoDB** deep-dive (data model, capacity, streams).



## Checklist hoàn thành Tuần 3

---

- ☐ Hoàn thành 4 buổi A/B/C/D
- ☐ Thuộc lòng công thức WCU/RCU + tự tính 5 kịch bản đúng
- ☐ Phân biệt rành GSI vs LSI (bảng so sánh)
- ☐ Hoàn thành toàn bộ Lab checklist (bảng + GSI + LSI + Streams → Lambda + conditional write + transaction)
- ☐ Ghi sổ câu sai + lên lịch spaced repetition 1/3/7 ngày
- ☐ Vượt Cổng tự kiểm tra